EPISODE 7

DSA with JavaScript

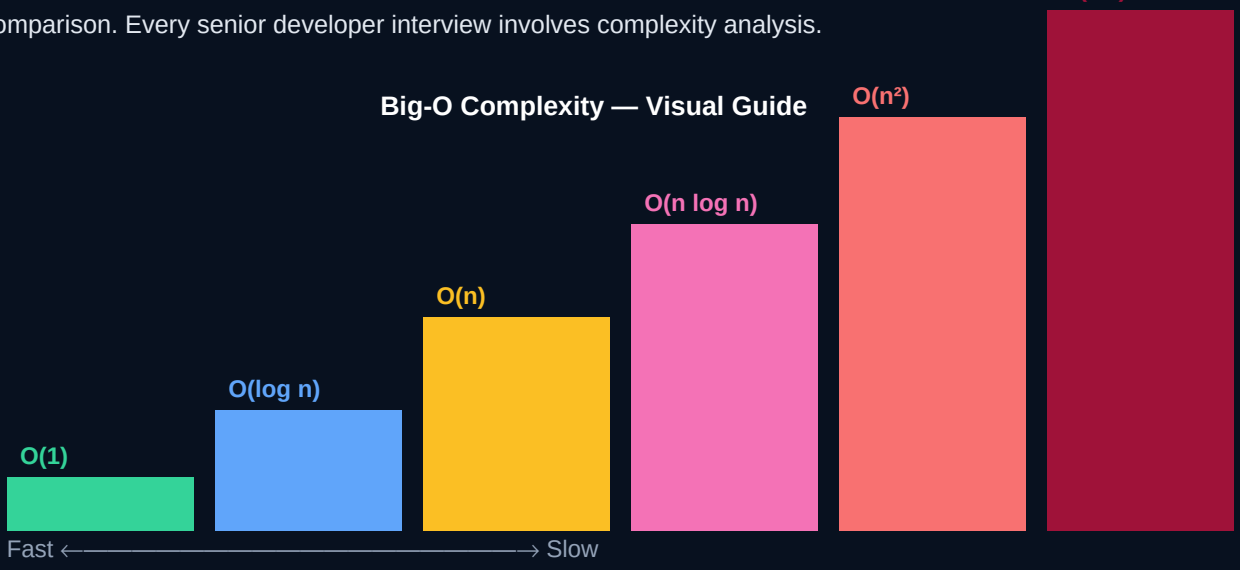
Data Structures, Algorithms & Complexity

Big-O Complexity	Arrays & Two Pointers
Sorting Algorithms	Binary Search
Linked List, Stack, Queue	Trees & BST

CHAPTER 1 — TIME & SPACE COMPLEXITY

Big-O Notation — Algorithmic Complexity

Big-O describes how an algorithm's runtime grows relative to input size. It's the language of algorithm comparison. Every senior developer interview involves complexity analysis.



Complexity comparison: O(1) is ideal, O(n²) is concerning, O(2■) is catastrophic for large n

Notation	Name	Example Algorithm	100 items → ops
O(1)	Constant	Array index access	1
O(log n)	Logarithmic	Binary search	7
O(n)	Linear	Linear search / single loop	100
O(n log n)	Linearithmic	Merge sort, Quick sort	664
O(n²)	Quadratic	Bubble sort, nested loops	10,000
O(2■)	Exponential	Fibonacci recursion (naive)	1.27 × 10³■

CHAPTER 2 — ARRAYS & TWO POINTERS

```
// Two-pointer technique — O(n) instead of O(n²)
function twoSum(nums, target) {
  let left = 0, right = nums.length - 1;  // sorted array required
  while (left < right) {
    const sum = nums[left] + nums[right];
    if (sum === target) return [left, right];
    if (sum < target) left++;
    else right--;
  }
  return [];
}

// Kadane's Algorithm — Maximum Subarray Sum — O(n)
function maxSubArray(nums) {
  let maxSum = nums[0], currentSum = nums[0];
  for (let i = 1; i < nums.length; i++) {
    currentSum = Math.max(nums[i], currentSum + nums[i]);
    maxSum     = Math.max(maxSum, currentSum);
  }
  return maxSum;
}

// Prefix Sum — range query in O(1) after O(n) build
function buildPrefix(nums) {
  const prefix = [0];
  for (const n of nums) prefix.push(prefix.at(-1) + n);
  return prefix;
}
// rangeSum(i, j) = prefix[j+1] - prefix[i]
```

CHAPTER 3 — SORTING ALGORITHMS

Sorting Algorithms Comparison

Bubble Sort	Selection Sort	Insertion Sort	Merge Sort	Quick Sort
Best: O(n²)	Best: O(n²)	Best: O(n)	Best: O(n log n)	Best: O(n log n)
Worst: O(n²)	Worst: O(n²)	Worst: O(n²)	Worst: O(n log n)	Worst: O(n²)
Space: O(1)	Space: O(1)	Space: O(1)	Space: O(n)	Space: O(log n)
Stable	Unstable	Stable	Stable	Unstable

Know Merge Sort and Quick Sort — they appear in 80% of sorting interview questions

```
// Merge Sort — O(n log n), stable
function mergeSort(arr) {
  if (arr.length <= 1) return arr;
  const mid   = Math.floor(arr.length / 2);
  const left  = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));
  return merge(left, right);
}
function merge(left, right) {
  const result = []; let i = 0, j = 0;
  while (i < left.length && j < right.length) {
    if (left[i] <= right[j]) result.push(left[i++]);
    else result.push(right[j++]);
  }
  return [...result, ...left.slice(i), ...right.slice(j)];
}

// Binary Search — O(log n)
function binarySearch(arr, target) {
  let left = 0, right = arr.length - 1;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] === target) return mid;
    if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1;
}
```

CHAPTER 4 — LINKED LIST, STACK & QUEUE

```
// Singly Linked List implementation
class Node { constructor(val) { this.val=val; this.next=null; } }
class LinkedList {
  constructor() { this.head=null; this.size=0; }
  append(val) {
    const node = new Node(val);
    if (!this.head) { this.head=node; }
    else { let curr=this.head; while(curr.next) curr=curr.next; curr.next=node; }
    this.size++;
  }
  reverse() {  // O(n)
    let prev=null, curr=this.head;
    while (curr) { const next=curr.next; curr.next=prev; prev=curr; curr=next; }
    this.head = prev;
  }
}

// Stack — LIFO (use array)
class Stack {
  #data = [];
  push(item) { this.#data.push(item); }
  pop()      { return this.#data.pop(); }
  peek()     { return this.#data.at(-1); }
  isEmpty()  { return this.#data.length === 0; }
}

// Valid Parentheses — classic stack problem
function isValid(s) {
  const stack = [], map = { ')':'(', ']':'[', '}':'{' };
  for (const ch of s) {
    if ('([{'.includes(ch)) stack.push(ch);
    else if (stack.pop() !== map[ch]) return false;
  }
  return stack.length === 0;
}
```

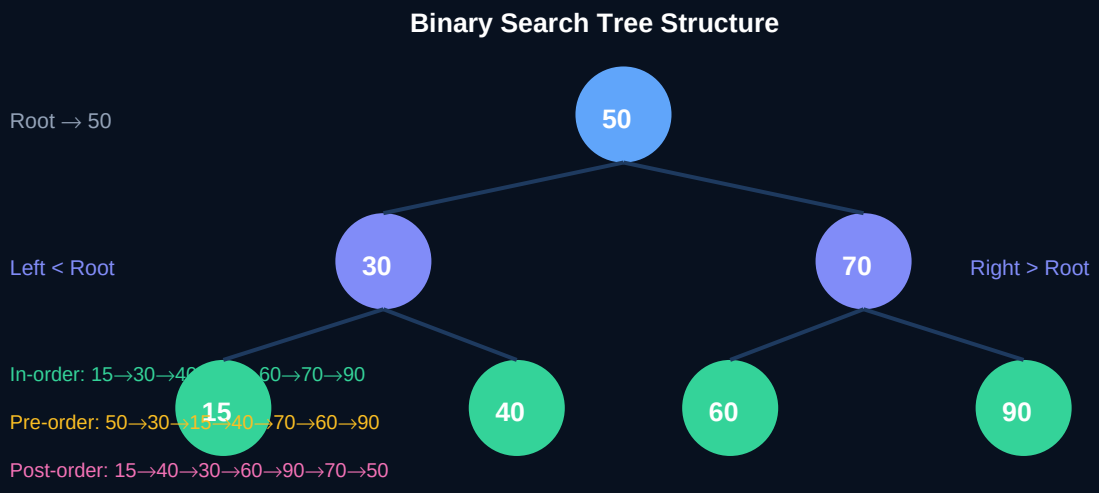
CHAPTER 5 — TREES & BINARY SEARCH TREE

Data Structures — Use Case Map

Array	Linked List	Stack	Queue
Index access	Insert/delete	LIFO	FIFO
Fixed-size	O(1) head	Undo/redo	BFS traversal
O(1) read	No random access	Function calls	Task queues

Hash Map	BST	Heap	Graph
O(1) lookup	Sorted data	Priority queue	Networks
Key-value pairs	O(log n) ops	O(log n) insert	Social graphs
Duplicates check	Range queries	Top-K problems	Path finding

Choose the right data structure — it's the difference between O(1) and O(n)



BST: left child < parent < right child — enables O(log n) search, insert, delete

```
// Binary Tree DFS traversals
function inorder(root, result=[]) {
  if (!root) return result;
  inorder(root.left, result);
  result.push(root.val);          // L → Root → R = sorted order
  inorder(root.right, result);
  return result;
}

// BFS — Level Order Traversal
function levelOrder(root) {
  if (!root) return [];
  const queue=[root], result=[];
  while (queue.length) {
    const levelSize = queue.length;
    const level = [];
    for (let i=0; i<levelSize; i++) {
      const node = queue.shift();
      level.push(node.val);
      if (node.left)  queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(level);
  }
  return result;
}
```

Episode 7 Summary — DSA Topics Covered

✓ Big-O Notation — O(1) through O(2■), practical complexity analysis
✓ Arrays — two-pointer, Kadane's algorithm, prefix sums
✓ Sorting — Merge Sort, Quick Sort, Binary Search
✓ Linked List — singly/doubly, reversal, fast/slow pointers
✓ Stack & Queue — LIFO/FIFO, monotonic stack, BFS with queue
✓ Hashing — Set and Map for O(1) lookup problems
✓ Trees & BST — DFS/BFS traversals, height, LCA, diameter
✓ Recursion & Backtracking — N-Queens, Sudoku, subsets